

Composition and conflicts

Rules for overlapping interventions

Interventions on different targets compose freely. Interventions on the same target compose only when their intervals are disjoint or the target declares an explicit combination algebra.

```
using CategoricalInterventions

beta = Target(:beta; kind=Parameter, value_type=Float64)

closure = InterventionAtom(:closure, beta, Interval(10, 30), Scale(0.6))
mandate = InterventionAtom(:mask_mandate, beta, Interval(20, 40), Scale(0.8))

program = InterventionProgram(closure, mandate)
conflicts(program)
```

```
1-element Vector{Conflict}:
```

```
Conflict(target=Parameter:beta, overlap=[20, 30), reason=no combination algebra for overlap)
```

By default, attempting to compose these two programs throws an `InterventionConflictError`, because both modify `:beta` on the overlap `[20, 30)`.

```
try
    compose_interventions(InterventionProgram(closure), InterventionProgram(mandate))
catch err
    typeof(err), sprint(showerror, err)
end
```

```
(InterventionConflictError, "Intervention program has 1 conflict(s):\n - Conflict(target=Pa
```

If overlapping effects are meant to combine multiplicatively, declare that on the target.

```

beta_mult = Target(
    :beta;
    kind=Parameter,
    value_type=Float64,
    algebra=MultiplicativeAlgebra(),
)

closure = InterventionAtom(:closure, beta_mult, Interval(10, 30), Scale(0.6))
mandate = InterventionAtom(:mask_mandate, beta_mult, Interval(20, 40), Scale(0.8))

program = compose_interventions(InterventionProgram(closure), InterventionProgram(mandate))
applied = apply_interventions(program, Dict(:beta => 0.30))

[(epoch.support, epoch.values[:beta]) for epoch in applied]

```

```

3-element Vector{Tuple{Interval{Int64}, Float64}}:
 ([10, 20], 0.18)
 ([20, 30], 0.144)
 ([30, 40], 0.24)

```

The overlap epoch applies both effects, so $0.30 * 0.6 * 0.8 = 0.144$. This makes the combination rule explicit rather than silently choosing an order.